

## 2.3.14

EE25BTECH11011 - Navya Banavathu

September 8,2025

# Question

If a line makes an angle of  $30^\circ$  with the positive direction of the X-axis,  $120^\circ$  with the positive direction of the Y-axis, then find the angle which it makes with the positive direction of the Z-axis

# Solution

Let the direction (unit) vector of the line be

$$d = \begin{bmatrix} a \\ b \\ c \end{bmatrix}. \quad (1)$$

Condition with  $x$ -axis:

$$d^T e_x = \cos 30^\circ \quad (2)$$

$$\begin{bmatrix} a & b & c \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \cos 30^\circ \quad (3)$$

$$a = \frac{\sqrt{3}}{2} \quad (4)$$

# Solution

Condition with  $y$ -axis:

$$d^T e_y = \cos 120^\circ \quad (5)$$

$$\begin{bmatrix} a & b & c \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \cos 120^\circ \quad (6)$$

$$b = -\frac{1}{2} \quad (7)$$

Now using normalization:

$$d^T d = 1 \quad (8)$$

$$\begin{bmatrix} a & b & c \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = 1 \quad (9)$$

# Solution

$$a^2 + b^2 + c^2 = 1 \quad (10)$$

$$\left(\frac{\sqrt{3}}{2}\right)^2 + \left(-\frac{1}{2}\right)^2 + c^2 = 1 \quad (11)$$

$$\frac{3}{4} + \frac{1}{4} + c^2 = 1 \quad (12)$$

$$c^2 = 0 \Rightarrow c = 0 \quad (13)$$

Hence,

$$d = \begin{bmatrix} \frac{\sqrt{3}}{2} \\ -\frac{1}{2} \\ 0 \end{bmatrix} \quad (14)$$

# Solution

Angle with z-axis:

$$d^T e_z = \cos \theta \quad (15)$$

$$\begin{bmatrix} \frac{\sqrt{3}}{2} & -\frac{1}{2} & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \cos \theta \quad (16)$$

$$0 = \cos \theta \quad (17)$$

$$\theta = 90^\circ \quad (18)$$

Therefore, the line makes angles of  $30^\circ$ ,  $120^\circ$ , and  $90^\circ$  with the x-, y-, and z-axes, respectively.

## C Program

```
#include <stdio.h>
#include <math.h>

#define PI 3.14159265358979323846

// Vector struct
typedef struct {
    double x, y, z;
} Vector3;

// Dot product of two vectors
double dotProduct(Vector3 a, Vector3 b) {
    return a.x * b.x + a.y * b.y + a.z * b.z;
}
```

## C Program

```
// Normalize a vector to unit length
Vector3 normalize(Vector3 v) {
    double length = sqrt(dotProduct(v, v));
    Vector3 result = {v.x / length, v.y / length, v.z / length};
    return result;
}

// Create vector from direction cosines (angles in degrees)
Vector3 fromDirectionCosines(double angle_x_deg, double
    angle_y_deg) {
    double a = cos(angle_x_deg * PI / 180.0);
    double b = cos(angle_y_deg * PI / 180.0);

    // Since  $a^2 + b^2 + c^2 = 1$ , solve for c
    double c_squared = 1.0 - (a*a + b*b);
    double c = 0.0;
```



# Code (C)

## C Program

```
if (c_squared >= 0) {
    c = sqrt(c_squared);
} else {
    printf("Warning: angles do not form a valid unit vector.\n");
}
Vector3 v = {a, b, c};
return normalize(v);
}

int main() {
    // Angles given in the problem
    double angle_x = 30.0; // degrees
    double angle_y = 120.0; // degrees

    Vector3 direction = fromDirectionCosines(angle_x, angle_y);
```

## C Program

```
printf("Unit direction vector:\n");
    printf("x = %.6f\n", direction.x);
    printf("y = %.6f\n", direction.y);
    printf("z = %.6f\n", direction.z);

// Compute angle with z-axis
Vector3 e_z = {0.0, 0.0, 1.0};
double cos_gamma = dotProduct(direction, e_z);
double gamma_deg = acos(cos_gamma) * 180.0 / PI;

printf("Angle with z-axis = %.2f degrees\n", gamma_deg);

return 0;
}
```

## Part 1

```
import ctypes
import math
import os
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# Load compiled C shared library
if os.name == "nt":
    lib = ctypes.CDLL("./direction.dll") # Windows
else:
    lib = ctypes.CDLL("./direction.so") # Linux/macOS

# Define Vector3 struct (same layout as in C)
class Vector3(ctypes.Structure):
    _fields_ = [("x", ctypes.c_double),
                ("y", ctypes.c_double),
                ("z", ctypes.c_double)]
```

## Part 2

```
# Function signatures
lib.fromDirectionCosines.argtypes = [ctypes.c_double, ctypes.c_double]
lib.fromDirectionCosines.restype = Vector3

lib.dotProduct.argtypes = [Vector3, Vector3]
lib.dotProduct.restype = ctypes.c_double

# Input angles
angle_x = 30.0
angle_y = 120.0

# Call C function to compute direction vector
direction = lib.fromDirectionCosines(angle_x, angle_y)
```

## Part 3

```
print("Unit direction vector (from C):")
print(f"x = {direction.x:.6f}")
print(f"y = {direction.y:.6f}")
print(f"z = {direction.z:.6f}")

# Compute angle with z-axis
e_z = Vector3(0.0, 0.0, 1.0)
cos_gamma = lib.dotProduct(direction, e_z)
gamma_deg = math.degrees(math.acos(cos_gamma))
print(f"Angle with z-axis = {gamma_deg:.2f} degrees")

# Example values for the direction cosines
cos_alpha = 0.866025
cos_beta = -0.5
cos_gamma = 0.0
```

# Python Code: Plotting

## Part 4

```
# Generate points along the line (both directions)
t = np.linspace(-3, 3, 100) # allows negative and positive values
x = cos_alpha * t
y = cos_beta * t
z = cos_gamma * t

# Create the plot
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

# Plot the direction line
ax.plot(x, y, z, color="blue", label=r"$L : (\cos\alpha, \cos\beta, \cos\gamma)t$")
```

## Part 5

```
# Draw X, Y, Z axes with arrows
ax.quiver(0, 0, 0, 3, 0, 0, color="red", arrow_length_ratio=0.05)
ax.text(3.2, 0, 0, "X", color="red")

ax.quiver(0, 0, 0, 0, 3, 0, color="green", arrow_length_ratio=0.05)
ax.text(0, 3.2, 0, "Y", color="green")

ax.quiver(0, 0, 0, 0, 0, 3, color="black", arrow_length_ratio=0.05)
ax.text(0, 0, 3.2, "Z", color="black")
```

## Part 5

```
# Axis labels and title
ax.set_xlabel("X-axis")
ax.set_ylabel("Y-axis")
ax.set_zlabel("Z-axis")
ax.set_title("Line with Direction Cosines")
ax.legend()

# Set view limits
ax.set_xlim([-3, 3])
ax.set_ylim([-3, 3])
ax.set_zlim([-3, 3])

# Save the figure
plt.savefig("direction_line_full.png", dpi=300, bbox_inches="tight")
plt.show()
```



# Graphical Representation

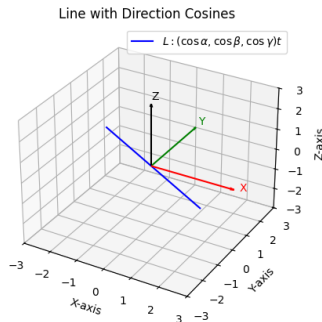


Figure: Direction Cosines